Universidad El Bosque

Ingeniería de Sistemas

Ingeniería de Software 2

Taller de pruebas unitarias y de integración

API de avistamientos de aves

Felipe Mejía Alvarado

Profesor: Jonathan Briceño

Bogotá, 21 de octubre de 2026

1. Qué cambió en el código

Antes, la validación estaba dentro del controlador, en un método llamado faltanDatos() que solo revisaba que los campos no llegaran vacíos. El problema es que para probarla tocaba levantar toda la API. Por eso hice tres cambios:

- **Saqué la validación a su propia clase**, ValidadorDeAvistamiento. Ahí está la regla del taller: la especie debe tener entre 3 y 50 caracteres (contados después de quitar los espacios de los extremos), la fecha no puede ser nula ni posterior a hoy, y el lugar y el observador no pueden estar vacíos. Si algo falla, devuelve un mensaje que dice qué campo está mal, y el controlador responde 400 con ese mensaje en el POST y el PUT.
- **Creé una interfaz Reloj** para obtener la fecha de hoy. La API usa RelojDelSistema, que devuelve la fecha real, pero en las pruebas puedo pasarle un reloj con una fecha fija.
- **Manejé el error de formato de la fecha.** Si llega una fecha como "hola", Spring falla al leer el JSON antes de llegar al validador, y la API respondía 400 sin decir qué campo falló. Agregué un manejador de ese error para que responda con el mensaje de la fecha.

Con esto la API es más fácil de probar: la validación se prueba creando un objeto y llamando un método, sin servidor ni base de datos. Las pruebas unitarias se demoran unos 0,02 segundos, mientras que las que levantan Spring tardan unos 3 segundos.

2. Tabla de casos

Primero dividí los valores posibles en particiones de equivalencia, es decir, grupos donde el validador se comporta igual:

Campo	Partición	¿Válida?
Especie	Nula (no se envía)	No
Especie	Menos de 3 caracteres (incluye vacía y solo espacios)	No
Especie	Entre 3 y 50 caracteres	Sí
Especie	Más de 50 caracteres	No
Fecha	Nula (no se envía)	No
Fecha	Mal escrita o que no existe ("hola", "20-10-2026")	No
Fecha	Hoy o antes	Sí
Fecha	Después de hoy	No

Tabla 1. Particiones de la especie y la fecha

Los espacios no son una partición aparte: como quito los espacios antes de contar, una especie de solo espacios queda con 0 caracteres y cae en "menos de 3".

Después elegí los valores. En los bordes de cada partición usé valores límite, porque ahí es donde más se cometen errores (por ejemplo, poner < en vez de <=):

Campo	Valor	Tipo	Resultado	Por qué este valor
Especie	"ab" (2)	Límite	Rechaza	Justo debajo del mínimo de 3.
Especie	"abc" (3)	Límite	Acepta	El mínimo exacto permitido.
Especie	50 caracteres	Límite	Acepta	El máximo exacto permitido.
Especie	51 caracteres	Límite	Rechaza	Justo encima del máximo.
Especie	"Águila"	Representativo	Acepta	Un nombre normal de la partición válida.
Especie	solo espacios	Inválido	Rechaza	Comprueba que los espacios no cuenten como letras.
Especie	null	Inválido	Rechaza	Que no se envíe el campo.
Fecha	hoy	Límite	Acepta	El último día permitido.
Fecha	mañana	Límite	Rechaza	El primer día no permitido.
Fecha	null	Inválido	Rechaza	Que no se envíe la fecha.
Fecha	"hola"	Inválido	400	Formato incorrecto; se prueba por HTTP (sección 3).
Lugar	vacío	Inválido	Rechaza	Que el lugar llegue vacío.
Observador	null	Inválido	Rechaza	Que no se envíe el observador.

Tabla 2. Casos de prueba

3. Por qué cada nivel

- **rechazaEspecie_cuandoTiene2Caracteres es unitaria** porque solo prueba la regla de la especie. No necesita la API ni la base de datos, es rápida y, si falla, sé que el error está en el validador.
- **crear_responde400_conMensaje_cuandoLaFechaTieneMalFormato es de integración** porque el error pasa al leer el JSON, en la frontera HTTP, antes de llegar al validador. Una prueba unitaria del validador nunca recibe el texto "hola".
- **crear_responde201_yLoGuarda_cuandoEsValido es de integración** porque revisa que todo funcione junto: la respuesta, el cuerpo y que el avistamiento sí quedó guardado en la base de datos.

Lo que la prueba de integración detecta y la unitaria no, me pasó durante el taller: las pruebas unitarias del validador pasaban todas, pero el controlador todavía usaba el método viejo y no llamaba al validador. Solo las pruebas de integración lo mostraron: un POST con la especie "ab" respondía 201 en vez de 400.

4. Los dobles

RelojFijo (stub). Reemplaza al reloj real y siempre dice que hoy es el 21 de octubre de 2026. Es un stub porque solo devuelve un valor fijo; no necesito revisar cómo lo llaman, que es lo que haría un mock. Sin este doble, la prueba de "fecha de mañana" dependería del día en que se corra: si pusiera 22 de octubre de 2026 a mano, pasaría hoy pero fallaría desde mañana.

Base de datos H2 en memoria (fake). En las pruebas de integración uso una base en memoria en vez del archivo real. Funciona igual pero es temporal, así que no ensucia los datos reales. Además, la limpio antes de cada prueba para que ninguna dependa de otra ni del orden en que se corran.

No usé un mock del repositorio porque la API no tiene capa de servicio, y en las pruebas de integración quería probar justamente el repositorio real.

5. Lo que no probé

- **Getters, setters y constructores del modelo:** no tienen lógica, así que una prueba solo comprobaría que Java guarda variables. Además, ya se usan en las pruebas de integración.
- **El método main():** solo arranca la aplicación, y las pruebas levantan Spring por su cuenta.
- **Una fecha que no existe, como "2026-02-30":** está en la misma partición que "hola" y la maneja el mismo código, así que no encontraría un error nuevo.
- **Lugar nulo y observador vacío:** el lugar y el observador usan el mismo método para revisar si están vacíos, y ese método ya queda probado con el lugar vacío y el observador nulo.

6. Lectura de la cobertura

Usé JaCoCo. En la primera ejecución, el total fue 91 % de instrucciones y 93 % de ramas, y el paquete de validación tenía 97 %.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.felipemejia.avistamientos.model		69%		n/a	4	12	8	23	4	12	0	1
com.felipemejia.avistamientos		37%		n/a	1	2	2	3	1	2	0	1
com.felipemejia.avistamientos.controller		100%		100%	0	12	0	22	0	7	0	1
com.felipemejia.avistamientos.validation		100%		100%	0	15	0	18	0	5	0	2
Total	21 of 262	91%	0 of 30	100%	5	41	10	66	5	26	0	5

Figura 1. Reporte final de cobertura de JaCoCo

Al revisar el reporte vi que en la validación faltaban dos cosas: la línea del error del observador y el caso en que un campo llega nulo (solo había probado el lugar vacío). Decidí agregar la prueba rechazaObservador_cuandoEsNulo porque era fácil y cubría las dos cosas a la vez. Con ella, la validación quedó en 100 % y el total de ramas pasó a 100 %.

No intenté cubrir el modelo (69 %) ni el método main() (37 %) por lo que expliqué en la sección 5. Como criterio de cobertura me fijé en las ramas de la lógica de negocio, no en el porcentaje total: llegar al 100 % en el modelo no me daría más confianza en la regla.

Las pruebas se corren con .\mvnw.cmd test (23 en total) y la cobertura con .\mvnw.cmd verify. El reporte queda en target/site/jacoco/index.html.